A Dockerfile is an automated build manifest containing step-by-step instructions that assemble a self-contained, reproducible virtual environment (called a Docker Container). In artificial intelligence and optical character recognition applications, Python code depends heavily on low-level operating system C++ binaries, graphics drivers (e.g. libgl1, libglib2.0-0), PyTorch C-extensions, and specific numerical libraries. Without containerization, moving code to a different machine frequently produces missing shared object errors or version conflicts.

The project Dockerfile is structured into six hardened stages:

Stage 1: Base Operating System

Directive: FROM python:3.10-slim

Explanation: Pulls an official lightweight (~150 MB) Debian Linux system with Python 3.10 pre-installed, minimizing container size and reducing attack surfaces.

Stage 2: Low-Level System Dependencies

Directive: RUN apt-get update && apt-get install -y --no-install-recommends libgl1 libglib2.0-0 curl && rm -rf /var/lib/apt/lists/*

Explanation: OpenCV requires Linux shared C++ libraries (libgl1 and libglib) to decode image matrices. Installing these via apt-get ensures cv2 never crashes with missing shared object errors.

Stage 3: Working Directory Definition

Directive: WORKDIR /app

Explanation: Sets the primary execution and storage folder inside the container to /app.

Stage 4: Dependency Caching & Installation

Directive: COPY requirements.txt . followed by RUN pip install --no-cache-dir -r requirements.txt

Explanation: Copies requirements.txt first and runs pip. If dependencies haven't changed, Docker reuses the cached layer, making future rebuilds take only seconds.

Stage 5: Pre-Baking Neural Network Weights

Directive: RUN python -c "import easyocr; easyocr.Reader(['en'], gpu=False)"

Explanation: Pre-downloads the CRAFT detector and CRNN recognition weights during the build step so end-users never experience runtime model download delays.

Stage 6: Port Exposure & Launch Entrypoint

Directive: EXPOSE 8501 | CMD ["streamlit", "run", "src/app.py", "--server.port=8501", "--server.address=0.0.0.0", "--server.headless=true"]

To eliminate the need for memorizing long terminal execution flags, docker-compose.yml stores all operational settings:

• Port Forwarding: Maps host machine port 8501 to container port 8501 ('8501:8501').

• Container Naming: Assigns a clean, human-readable name ('ocr_detection_service').

• Restart Policy: Set to 'unless-stopped' so the container automatically recovers if the host reboots.

• Healthcheck: Periodically pings Streamlit's internal health endpoint (/_stcore/health) to ensure high availability.

Follow these exact steps to launch, inspect, and verify the container on your system:

Step 4.1: Launch Docker Desktop

• Open Docker Desktop from the Windows Start Menu.

• Verify the bottom-left status indicator turns GREEN and displays 'Engine running'.

Step 4.2: Execute the Launch Command in Terminal

• Open Command Prompt or PowerShell.

• Navigate to: cd C:\Pankaja\AUTOMATED_SDLC_AGENT_PROJECTS\ocr_detection

• Execute the build and run command:

docker-compose up -d --build

• Docker builds the layers and reports: 'Container ocr_detection_service Started'.

Step 4.3: Terminal Status Verification

• In terminal, run: docker ps

- **Confirm the output shows:**

  - IMAGE: ocr-detection:1.0.0

  - STATUS: Up X seconds / minutes

  - PORTS: 0.0.0.0:8501->8501/tcp

  - NAMES: ocr_detection_service

Step 4.4: Web Browser Functional Verification

- **Open Google Chrome, Microsoft Edge, or Firefox.**
- **Navigate to: http://localhost:8501**
- **Functional Tests to Execute:**

  1. Click '🚘 Vehicle Number Plate' -> Verify bounding box extracts 'MH 12 AB 1234'.

  2. Click '🧾 Store Receipt' -> Verify line items and totals extracted.

  3. Click '📄 Printed Document' -> Verify full document text extracted.

  4. Click 'Download as CSV' -> Verify extracted_table.csv downloads to your computer.

Step 4.5: Internal Automated Test Execution Inside Docker

- **To run the automated 26-point pytest suite directly inside the Linux container, execute:**

  docker exec -it ocr_detection_service pytest tests/

- **Start container in background: docker-compose up -d**
- **Rebuild and restart after updates: docker-compose up -d --build**

• View real-time container logs: docker logs -f ocr_detection_service

• Stop container gracefully: docker-compose down

• Inspect container resource usage (CPU/RAM): docker stats ocr_detection_service